

CloudOrders

Guia de Estudo de Arquitetura Distribuída

Do walking skeleton à Transactional Outbox — o sistema implementado antes da fase de deployment em Azure

Material de estudo baseado no repositório CloudOrders

Um roteiro técnico, em português brasileiro, para estudar o sistema CloudOrders tal como ele está implementado antes da fase de deployment em Azure.

Como usar este guia

Este livro não é uma especificação nova nem uma receita genérica de microserviços. Ele organiza, em uma única narrativa, as decisões que já existem no CloudOrders: a API de pedidos, PostgreSQL, Azure Service Bus, três workers, Inbox/idempotência, retry/DLQ, OpenTelemetry e Transactional Outbox. Quando algo ainda não existe — deployment, CI/CD, IaC, replay de DLQ, limpeza de Outbox, integração com provedores reais ou coordenação de múltiplos dispatchers — ele aparece como próximo passo, nunca como capacidade pronta.

O melhor modo de estudar é percorrer os capítulos em ordem e, em paralelo, abrir o código e os testes citados pelo nome. Os capítulos em 01 a 10 deste diretório aprofundam cada estágio histórico; este guia conecta os estágios e prioriza as relações de causa e efeito. Os contratos canônicos em openspec/specs, as ADRs em docs/decisions e o README.md são referências de apoio para a implementação.

Estado coberto

O ponto de partida é um POST /orders. O ponto de chegada é a publicação eventual de OrderCreated no tópico order-events, seu fan-out para Payment, Inventory e Notification, e o processamento idempotente com settlement explícito. A API persiste o pedido e a mensagem de saída na mesma transação PostgreSQL; um dispatcher hospedado publica depois. A aplicação entrega semântica **at-least-once** de publicação e entrega, com efeitos internos de cada consumidor protegidos pelo Inbox. Não há garantia universal de exactly-once.

Sumário

- 1 Fundação: walking skeleton e limites
- 2 Persistência: PostgreSQL, EF Core e testes
- 3 Mensageria: eventos, tópicos e contratos
- 4 Fan-out: workers e processamento
- 5 Falhas: retry, redelivery e DLQ
- 6 Inbox: idempotência no consumidor
- 7 Observabilidade: traces, métricas e logs
- 8 Outbox: publicação confiável
- 9 O sistema inteiro: garantias e evolução
- 10 Referência para revisão

1. Fundação: walking skeleton e limites

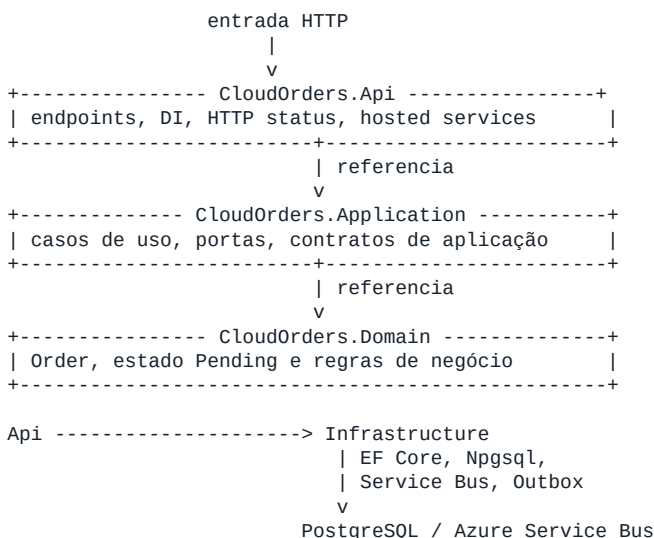
O problema e a primeira solução possível

Uma arquitetura distribuída pode ser desenhada por meses sem provar que as decisões se encaixam. Uma abordagem ingênua seria criar logo uma coleção de camadas, projetos compartilhados, interfaces genéricas, brokers e abstrações para casos futuros. Ela parece preparada, mas posterga a única pergunta importante: uma requisição real consegue atravessar o sistema, aplicar uma regra e persistir um resultado?

O CloudOrders começou com um **walking skeleton**: o menor caminho vertical útil, executável de ponta a ponta. No estado inicial, `POST /orders` atravessava API, Application, Domain e Infrastructure até PostgreSQL. Não existiam ainda eventos de negócio, Service Bus, workers, Inbox, observabilidade ou Outbox. Essa distinção histórica importa: os elementos posteriores foram adicionados quando havia um problema concreto para resolver.

Limites da solução

O esqueleto tornou visíveis responsabilidades que continuam válidas no estado atual:



Workers --> Application + Infrastructure --> Service Bus / PostgreSQL

`CloudOrders.Api` é a borda HTTP: recebe a requisição, converte-a para o caso de uso e devolve a resposta. `CloudOrders.Application` orquestra o que o sistema faz e define portas como `IOrderStore`, `IMessagePublisher` e stores de Outbox/Inbox. `CloudOrders.Domain` contém o modelo de negócio, sem saber de HTTP, EF Core ou Azure. `CloudOrders.Infrastructure` materializa as portas com PostgreSQL, EF Core/Npgsql e Service Bus. Os workers têm borda de transporte própria, mas usam Application e Infrastructure; eles não colocam regra de negócio dentro da camada Azure.

A direção da dependência evita que o domínio passe a depender de uma escolha de transporte ou banco. Isso não é uma busca estética por “camadas perfeitas”: permite testar uma regra sem broker, substituir um adaptador e identificar onde uma falha pertence. A inversão não elimina acoplamento; ela o localiza nos pontos onde ele é necessário.

Arquitetura não é abstração em excesso

No começo, uma interface especializada era útil quando representava uma operação do sistema. `IOrderStore`, por exemplo, expressa persistir um pedido e, no estado atual, a operação especializada que grava `Order` com `OutboxMessage` na mesma transação. Um repositório genérico como `IRepository<T>` esconderia operações relevantes atrás de `Add`, `Get` e `Save`, sem reduzir a dependência real de EF Core nem explicar uma transação de negócio.

Pelo mesmo motivo, não nasceu um framework genérico de workers. `Payment`, `Inventory` e `Notification` compartilham convenções de transporte, porém têm identidade de consumidor, validação e simulação próprias. Uma abstração só deve ser extraída quando a repetição revelar um comportamento estável que vale padronizar. Antecipar tudo cria extensão para cenários imaginários e dificulta aprender o fluxo verdadeiro.

O que o caminho inicial provou

O walking skeleton validou a composição de projetos, injeção de dependência, endpoint, caso de uso, mapeamento EF Core e PostgreSQL. Depois, esse caminho permitiu acrescentar mensageria sem transformar o endpoint em produtor de efeitos externos síncronos; depois permitiu workers; depois exigiu idempotência; e finalmente levou à Outbox para resolver o dual-write. A sequência não é acidental: cada camada de confiabilidade respondeu a uma falha que a anterior revelou.

Key takeaways

- Walking skeleton é um fluxo mínimo vertical que prova integração, não uma versão “incompleta” de uma arquitetura final.
- API, Application, Domain e Infrastructure têm responsabilidades diferentes e dependências direcionadas.
- Uma abstração é útil quando preserva uma operação de negócio; abstração genérica preventiva tende a esconder o que importa.

Common mistakes

- Dizer que Service Bus ou Outbox existiam no skeleton original.
- Colocar regra de pedido no endpoint ou lógica de settlement no processador de negócio.
- Confundir “mais interfaces” com “melhor arquitetura”.

Revisão

- 1 **Por que iniciar por um walking skeleton?** Para provar o caminho real de valor e expor decisões de integração cedo.
- 2 **Onde fica o mapeamento para PostgreSQL?** Em Infrastructure, por meio de EF Core/Npgsql e `CloudOrdersDbContext`.
- 3 **Domain pode depender de Azure Service Bus?** Não; isso inverteria a direção e acoplaria regra de negócio ao transporte.

2. Persistência: PostgreSQL, EF Core e testes

Por que PostgreSQL é fundacional

Pedido, Inbox e Outbox precisam de transações, restrições e persistência relacional previsível. PostgreSQL foi escolhido como banco primário do CloudOrders, e EF Core com Npgsql é a implementação de acesso. CloudOrdersDbContext expõe os conjuntos de pedidos, mensagens processadas e mensagens de saída. Essa escolha dá ao projeto um banco real para a execução local e para testes de integração, em vez de uma simulação em memória que não reproduziria SQL, chaves compostas, ON CONFLICT nem migrations.

Uma alternativa ingênua é subir a aplicação e chamar `Database.Migrate()` no startup. Isso parece conveniente, mas mistura a inicialização de uma instância com uma alteração de schema que pode ser destrutiva, concorrente ou sem aprovação. CloudOrders usa **migrations explícitas**: as migrations são versionadas e aplicadas deliberadamente. A aplicação não as executa automaticamente ao iniciar. Em ambiente local, a pessoa desenvolvedora aplica a migration; nos testes de integração, a fixture prepara um banco efêmero e chama a migração como parte controlada do ambiente de teste.

Do modelo ao banco

Order representa o pedido do domínio. O DbContext mapeia a entidade e as tabelas de suporte. Há detalhes que são arquiteturalmente relevantes: `processed_messages` tem chave composta (`consumer_name`, `message_id`); `outbox_messages` persiste o payload e metadados necessários para nova tentativa; `MessageId` de saída é único. Essas invariantes pertencem ao schema porque concorrência e reinicialização não podem depender só da memória de um processo.

IOrderStore não é um repositório genérico. Ele é uma porta orientada ao caso de uso: hoje inclui uma operação que persiste pedido e Outbox juntos. A implementação EfOrderStore abre uma transação local, adiciona os dois registros, salva e confirma. Essa especialização revela o limite transacional; um `Repository<Order>` genérico não explicaria por que a operação também precisa gravar um evento de saída.

Desenvolvimento local e Testcontainers

O `compose.yaml` sobe PostgreSQL para uso manual, com volume nomeado `cloudorders_pg`. Um volume é deliberadamente separado do container: recriar o container não destrói os dados inicializados. Isso é produtivo para desenvolvimento, mas traz uma armadilha: trocar uma variável de senha no Compose não muda a senha gravada pelo PostgreSQL em um diretório de dados que já foi inicializado. A configuração inicial só é usada quando o volume nasce. Para reinicializar credenciais, é preciso agir conscientemente sobre o volume ou usar um ambiente novo; nunca se deve tentar “corrigir” o problema expondo segredos em documentação.

```
Desenvolvimento manual
+-----+
| Visual Studio | -----> | Docker Compose: PostgreSQL |
| API + migrations |      | volume cloudorders_pg |
+-----+
+-----+

Teste de integração
+-----+
| xUnit/Testcontainers | -----> | PostgreSQL efêmero 17 |
| aplica migrations |      | isolado por execução |
+-----+
+-----+
```

Testcontainers não depende do banco que alguém deixou no Compose. Ele cria PostgreSQL isolado, aplica migrations e verifica o comportamento contra uma instância real. “Funciona no meu Compose” é evidência de uma configuração manual específica; não demonstra que o schema atual, as migrations ou a transação serão corretos

numa execução limpa. Já o teste de integração reduz esse viés e detecta tabelas ausentes, migration esquecida e SQL que só falha em PostgreSQL.

Para configuração, User Secrets servem ao desenvolvimento local e variáveis de ambiente servem bem a execução hospedada. Em .NET, provedores de configuração posteriores podem sobrescrever valores anteriores; a ordem efetiva precisa ser entendida antes de diagnosticar uma connection string. O guia evita credenciais reais: use placeholders e inspecione qual origem de configuração ganhou, nunca copie senhas para logs ou commits.

Falhas e trade-offs

Erro de autenticação normalmente aponta para host, usuário/senha ou volume inicializado com credencial anterior; tabela inexistente aponta para migration não aplicada; divergência entre modelos e schema pode indicar migration ausente ou banco antigo. O diagnóstico começa com o erro do Npgsql, a connection string sem segredo e o histórico de migrations, não com alterações aleatórias de código.

Compose traz baixo atrito e estado persistente, mas exige disciplina de reset e de segredos. Testcontainers traz isolamento e fidelidade, mas consome Docker e tempo de inicialização. A combinação é intencional: um é ambiente de exploração manual; o outro é evidência repetível de persistência.

Key takeaways

- PostgreSQL real sustenta transações, constraints e SQL que o projeto precisa estudar.
- Migrations explícitas evitam mudança de schema surpresa no startup.
- Compose é útil para trabalho manual; Testcontainers verifica o caminho em banco limpo e isolado.

Common mistakes

- Achar que mudar senha no Compose altera um volume já inicializado.
- Confundir erro de migration com erro de credencial.
- Tratar IOrderStore como fachada para um CRUD genérico.

Revisão

- 1 **Por que não migrar no startup?** Porque evolução de schema é uma operação explícita, não efeito colateral de iniciar uma instância.
- 2 **O que garante a chave de Inbox?** O banco, pela chave composta (ConsumerName, MessageId).
- 3 **Por que Testcontainers complementa Compose?** Ele elimina estado manual e valida migrations e SQL contra PostgreSQL real.

3. Mensageria: eventos, tópicos e contratos

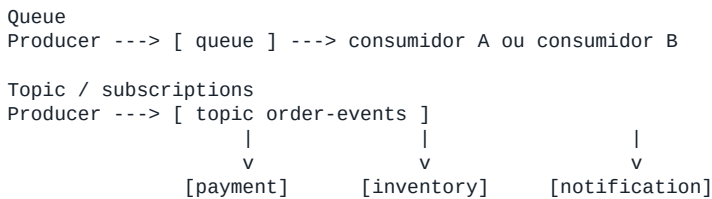
Evento de negócio e contrato de fio

`OrderCreated` expressa um fato de negócio: um pedido foi criado. Seu contrato contém apenas dados de negócio/evento — `OrderId`, `CustomerId`, `Status` e `CreatedAtUtc`. O status é serializado explicitamente como `Pending`; não depende cegamente de `enum.ToString()`, cujo nome poderia mudar por refatoração e alterar o contrato externo de modo involuntário.

O payload JSON não replica metadados do transporte. `MessageId` identifica logicamente o envelope publicado; `CorrelationId`, para esse evento, recebe o `OrderId` em formato D e permite relacionar mensagens de uma mesma ordem; `Subject` é `CloudOrders.Orders.OrderCreated`; `ContentType` é JSON. `TraceId` é identidade técnica de observabilidade W3C, não dado de negócio. Dessa forma, o contrato permanece estável mesmo se a infraestrutura de tracing evoluir.

Queue versus tópico

Uma queue distribui uma mensagem a um consumidor lógico. Um tópico com subscriptions mantém a publicação única e permite que cada subscription receba sua própria cópia lógica, filtrada. `CloudOrders` escolheu o tópico `order-events` porque um pedido criado interessa, de modo independente, a `Payment`, `Inventory` e `Notification`. O primeiro uso de Service Bus, historicamente, foi a probe de desenvolvimento: ela demonstrou conectividade, publicação, filtro e consumo antes que um evento de pedido existisse.



No tópico, subscriptions usam filtro SQL sobre `sys.Label`, que corresponde ao `Subject` da mensagem. Quando se cria filtro explícito, o filtro padrão `TrueFilter` precisa ser removido; caso contrário, ele aceita tudo e anula a intenção do filtro específico. A subscription `messaging-probe` usa outro subject; as subscriptions de negócio filtram `CloudOrders.Orders.OrderCreated`.

Service Bus Standard é necessário para tópicos e subscriptions neste projeto. A escolha é suficiente para o laboratório e evita custo e desenho de capacidades não usadas. O provisionamento documentado é manual no README, não IaC nem uma implantação Azure já feita.

Responsabilidades e garantias

O produtor define um contrato bem formado, metadados coerentes e publica no tópico. O consumidor valida contrato, aplica sua regra, persiste seu progresso quando necessário e decide settlement. O broker entrega com semântica `at-least-once`: uma mensagem pode reaparecer; ele não garante um efeito de negócio exatamente uma vez em sistemas externos.

Em `PeekLock`, o worker recebe um lock temporário. Com `AutoCompleteMessages=false`, ele chama `CompleteMessageAsync` só depois de concluir processamento; em falha transitória, deixa a mensagem sem settlement para redelivery. Isso separa “o broker entregou” de “o consumidor pode afirmar que terminou”.

Key takeaways

- OrderCreated carrega fato de negócio; metadados nativos carregam identidade de transporte.
- Tópico/subscriptions habilita fan-out independente; queue não é a mesma topologia.
- Service Bus oferece entrega at-least-once, não exactly-once de efeito de negócio.

Common mistakes

- Duplicar MessageId, CorrelationId ou trace context no JSON sem necessidade.
- Deixar o TrueFilter padrão junto ao filtro explícito.
- Confundir CorrelationId com TraceId.

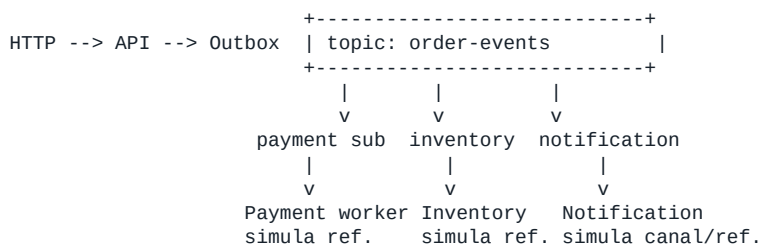
Revisão

- 1 **Por que OrderId é a correlação de negócio?** Porque identifica a ordem a que o evento se refere, independentemente de tentativas técnicas.
- 2 **Por que o MessageId é estável?** Para que publicação duplicada e entrega repetida representem a mesma mensagem lógica.
- 3 **O que a probe comprovou?** A infraestrutura de mensageria antes da introdução de eventos de negócio.

4. Fan-out: workers e processamento

Um evento, três interesses independentes

Uma publicação de `OrderCreated` chega a três subscriptions: `Payment`, `Inventory` e `Notification`. Cada worker é separado porque representa um consumidor independente, com ciclo de falha, observabilidade e futura integração externa próprios. Falhar ao reservar estoque não deve impedir a notificação de receber sua cópia, nem uma falha de pagamento deve travar a subscription de `inventory`.



O wrapper de cada worker conhece `ServiceBusProcessor`, `PeekLock`, bytes da mensagem, metadados nativos e delegates de completar/dead-letter. O processador conhece o contrato `OrderCreated`, validação e a simulação de negócio. Essa separação impede que o processador dependa de SDK Azure para testes unitários e torna explícito o que é transporte e o que é decisão da aplicação.

Caminho de sucesso de um consumidor

- 1 A subscription entrega a cópia em `PeekLock`.
- 2 O wrapper encaminha corpo, `MessageId`, `CorrelationId`, `Subject`, `ContentType` e `DeliveryCount` ao processador.
- 3 O processador valida envelope e JSON; então confirma que o status de negócio é `Pending`.
- 4 Dentro do `Inbox`, tenta reivindicar a mensagem para sua identidade de consumidor.
- 5 No primeiro processamento, executa a simulação determinística e marca a entrada como concluída na transação local.
- 6 O wrapper chama `CompleteMessageAsync`.

No estado atual, `Payment` gera `simulated-payment-{OrderId:N}`, `Inventory` gera `simulated-reservation-{OrderId:N}` e `Notification` usa o canal `simulated` com `simulated-notification-{OrderId:N}`. Determinismo permite que testes esperem o mesmo resultado para a mesma ordem e deixa clara a fronteira: não são cobranças, reservas nem notificações reais.

`MaxConcurrentCalls=1` foi escolha didática inicial. Facilita ler logs, reproduzir ordem e estudar locks e idempotência antes de introduzir concorrência local. Não é uma alegação de throughput de produção. A capacidade real seria reavaliada com medição, locks, duração de processamento e limite de cada provedor.

Falhas por categoria

Falha de **transporte** é receber, renovar lock ou fazer settlement no broker. Falha de **contrato** é `MessageId` ausente, `subject/content type` inesperado ou JSON inválido. Falha de **regra de negócio** neste estágio inclui status não suportado. As duas últimas classes permanentes vão imediatamente à DLQ com razão apropriada; falhas transitórias de PostgreSQL ou processamento ficam unsettled para redelivery.

Nenhum “worker framework” genérico foi criado. O que é comum é o padrão de integração deliberado; o que varia continua explícito nos três projetos. Quando houver provedores reais, cada consumidor precisará de timeout, autenticação, reconciliação e, provavelmente, uma chave de idempotência no provedor. O Inbox local não transforma uma chamada externa em exactly-once.

Key takeaways

- Fan-out isola consumidores: cada subscription tem entrega, retry e DLQ próprios.
- Wrapper resolve transporte; processador resolve contrato, negócio e uso de Inbox.
- As referências atuais são simulações determinísticas, não integrações externas.

Common mistakes

- Achar que uma falha em payment impede inventory e notification.
- Usar DeliveryCount como se fosse MessageId.
- Interpretar MaxConcurrentCalls=1 como recomendação final de escala.

Revisão

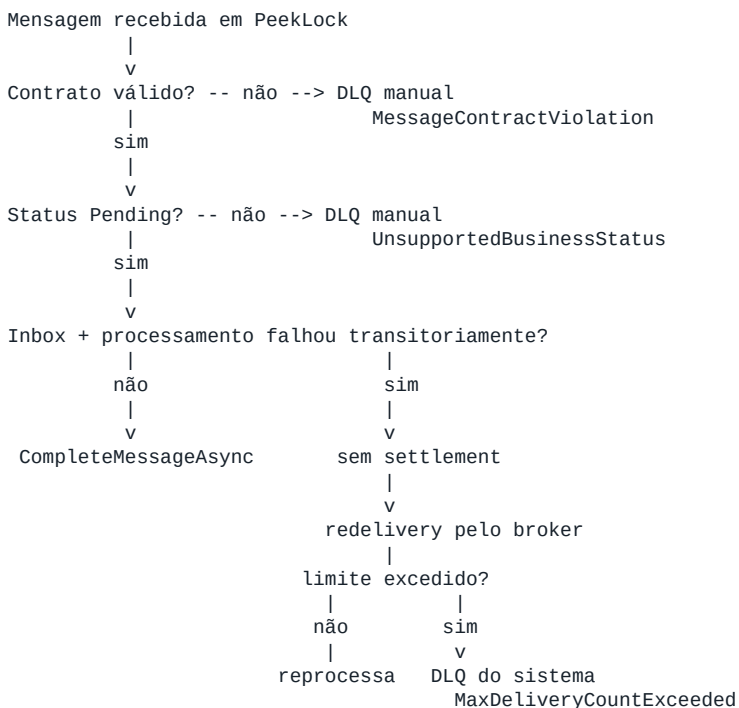
- 1 **Por que há três workers?** Porque os três efeitos pertencem a consumidores independentes de um mesmo fato.
- 2 **Quando completar a mensagem?** Após o processamento e a transação de Inbox terminarem com sucesso.
- 3 **O que é simulado?** Ação e referências de pagamento, reserva e notificação, não o transporte nem a persistência de idempotência.

5. Falhas: retry, redelivery e DLQ

Não são sinônimos

Retry local é o mesmo processo tentar novamente sem devolver controle ao broker. **Redelivery** é uma nova entrega pelo Service Bus de uma mensagem que não foi completada. **Dead-lettering** move uma mensagem que não deve ou não consegue progredir para a DLQ. CloudOrders, nesta fase, não usa Polly nem loop local para processamento de mensagem. Ao encontrar falha transitória, lança a exceção e não faz settlement; o Service Bus controla a redelivery conforme o lock e a política da subscription.

Essa escolha evita multiplicar políticas concorrentes e mantém o histórico de tentativas no broker. Para subscriptions de negócio, o provisionamento documentado usa `MaxDeliveryCount=5`. Isso é configuração de recurso Azure descrita no README, não uma constante criada pelo código local e tampouco prova que um recurso remoto já esteja implantado.



Classificação prática

Tipo de falha	Ação	Resultado esperado
Contrato inválido	Dead-letter imediato	DLQ: MessageContractViolation
Status de negócio não suportado	Dead-letter imediato	DLQ: UnsupportedBusinessStatus
PostgreSQL indisponível/transitório	Exceção, sem settlement	Redelivery; depois DLQ do broker
Falha de processamento transitória	Exceção, sem settlement	Redelivery; depois DLQ do broker
Falha em CompleteMessageAsync	Exceção/settlement pendente	Pode haver nova entrega

O projeto evita `AbandonMessageAsync` manual como mecanismo de retry. A falta de completion já permite a redelivery sem introduzir outra semântica de tentativa. Falha de settlement e expiração de lock também são falhas técnicas: mesmo que o banco tenha confirmado o trabalho, a mensagem pode voltar. É precisamente por isso que o Inbox existe.

Uma mensagem poison é aquela que não progride sem intervenção: contrato que nunca poderá ser lido, status que o consumidor não suporta ou falha repetida. Descrição de DLQ deve ser útil para diagnosticar — categoria e contexto seguro — sem payload bruto, connection string ou segredo. Ferramenta de replay, política de backoff avançada, retry diferenciado por dependência e operação de limpeza da DLQ permanecem adiados.

Key takeaways

- CloudOrders usa redelivery do broker, não retry local, para falhas transitórias nesta fase.
- Falhas permanentes conhecidas recebem DLQ explícita; transitórias chegam à DLQ após o limite do broker.
- Settlement pode falhar depois do commit de negócio; o consumidor precisa tolerar isso.

Common mistakes

- Chamar qualquer nova tentativa de “retry” sem indicar se é local ou redelivery.
- Mandar indiscriminadamente toda exceção à DLQ manual.
- Guardar payload ou segredos na descrição de dead-letter.

Revisão

- 1 **Por que não usar Polly aqui?** Para não combinar loops locais com a política de entrega já controlada pelo broker no estágio de aprendizagem.
- 2 **Quando é DLQ imediata?** Em violação permanente de contrato ou status de negócio não suportado.
- 3 **Como uma falha de PostgreSQL chega à DLQ?** A mensagem fica unsettled, é redeliverada e, ao exceder o limite configurado, o broker a move.

6. Inbox: idempotência no consumidor

Duplicata de mensagem versus efeito duplicado

At-least-once significa que a mesma mensagem lógica pode ser entregue mais de uma vez. Isso não é ainda um efeito de negócio duplicado; torna-se um problema quando o consumidor executa novamente uma ação persistente ou externa. A idempotência é responsabilidade do consumidor, pois somente ele sabe o que significa “já processei este efeito”.

CloudOrders usa o **Inbox Pattern** em `processed_messages`. Cada consumidor tem uma identidade: Payment, Inventory e Notification podem processar legitimamente o mesmo MessageId, cada um uma vez. Por isso a chave é (ConsumerName, MessageId), e não apenas MessageId. A mesma publicação deve alcançar os três; ela não deve gerar duas ações no mesmo consumidor.

Reivindicação atômica

Checar se existe e depois inserir é vulnerável a corrida: duas entregas concorrentes podem observar a ausência antes de qualquer uma inserir. A implementação EF usa `INSERT ... ON CONFLICT DO NOTHING RETURNING 1` dentro da transação. O retorno revela, atomicamente, quem ganhou a reivindicação. Só quem inseriu executa a ação de negócio; ao fim bem-sucedido, o registro chega a `Completed` e a transação confirma.

```
Primeira entrega
Broker -> Worker: mensagem M em PeekLock
Worker -> PostgreSQL: INSERT (consumer, M) ON CONFLICT DO NOTHING
PostgreSQL --> Worker: claim adquirido
Worker -> PostgreSQL: simulação + estado Completed, COMMIT
Worker -> Broker: CompleteMessageAsync
```

```
Duplicata após conclusão
Broker -> Worker: mesma mensagem M
Worker -> PostgreSQL: INSERT ... ON CONFLICT DO NOTHING
PostgreSQL --> Worker: claim não adquirido / Completed existente
Worker -> Broker: CompleteMessageAsync, sem repetir efeito
```

Se processamento de negócio falha, a transação local é revertida: não há `Completed`, e a redelivery pode tentar novamente. A janela crítica é outra: negócio e Inbox confirmam, mas `CompleteMessageAsync` falha. A mensagem volta; o Inbox encontra a conclusão e pula o efeito, enquanto o worker tenta apenas o settlement de novo. Esse é um caso correto e esperado em sistemas distribuídos.

```
Commit de negócio e Inbox --> falha ao completar no broker
```



Limites

Inbox não é transação distribuída com um provedor externo. Se uma futura chamada a adquirente, estoque real ou e-mail confirmar fora do PostgreSQL antes de uma falha local, o sistema precisará de chave idempotente no provedor, reconciliação, registro de estado e tratamento de ambiguidade. Inbox protege o efeito local que o CloudOrders implementa agora e complementa a Outbox: a primeira controla consumo duplicado; a segunda reduz perda entre banco e broker.

Key takeaways

- Mensagens duplicadas são normais sob at-least-once; efeitos duplicados não precisam ser.
- (ConsumerName, MessageId) preserva fan-out e bloqueia repetição no mesmo consumidor.
- ON CONFLICT DO NOTHING RETURNING 1 substitui uma verificação concorrente insegura.

Common mistakes

- Usar apenas MessageId como chave de Inbox e impedir os outros workers.
- Considerar Completed antes de confirmar a transação.
- Afirmar que Inbox dá exactly-once para um provedor externo.

Revisão

- 1 **Por que a chave é composta?** Porque cada consumidor tem seu próprio efeito legítimo para a mesma mensagem.
- 2 **O que ocorre se a regra falha?** A transação de Inbox é revertida e a mensagem pode ser redeliverada.
- 3 **O que ocorre se completion falha após commit?** Redelivery encontra o Inbox concluído e evita repetir o efeito.

Falha de autenticação PostgreSQL aparece em spans/logs de conexão e aponta para credencial, origem da configuração ou volume existente. Falha de tabela/migration aparece após a conexão e aponta para schema. Ambas não são “erro de worker” genérico e exigem respostas diferentes. Logs devem ser estruturados e não carregar corpo bruto de mensagem, connection strings ou segredos.

Key takeaways

- Logs, traces e métricas respondem perguntas complementares.
- TraceId é técnico; OrderId, MessageId e CorrelationId têm semântica de negócio/transporte distinta.
- Métricas devem ter baixa cardinalidade e telemetria precisa preservar dados sensíveis.

Common mistakes

- Usar IDs de pedido como tags de métrica.
- Inferir processamento de mensagem apenas por métricas de runtime.
- Colocar trace context no contrato OrderCreated.

Revisão

- 1 **O que identifica cada worker em telemetria?** Seu `service.name` próprio.
- 2 **Por que a Outbox guarda `traceparent`?** Para propagar contexto técnico até o dispatch sem contaminar o payload de negócio.
- 3 **O que diferencia `auth failure` de `schema failure`?** A primeira falha ao conectar/autenticar; a segunda conecta, mas não encontra o schema esperado.

8. Outbox: publicação confiável

O dual-write original

Antes da Outbox, o fluxo histórico persistia o pedido e depois publicava `OrderCreated`. Se o commit terminasse e o Service Bus falhasse, havia pedido sem evento. Inverter a ordem criaria evento para pedido inexistente se o banco falhasse. Uma transação PostgreSQL local não engloba Azure Service Bus: não existe atomicidade distribuída automática entre esses dois recursos no desenho do CloudOrders.

```
Fluxo antigo (inseguro)
POST -> grava Order -> COMMIT -> publica no Service Bus
      |               |
      |               +-- falha: pedido sem evento
      +-- sucesso
```

Transactional Outbox troca o ponto de publicação: no mesmo commit local que cria `Order`, cria uma `OutboxMessage`. Assim, a API pode devolver `201 Created` após o commit PostgreSQL; ela confirma a aceitação durável do pedido e da intenção de publicar, não a entrega já realizada no broker.

```
Fluxo atual (atômico localmente)
POST -> cria Order + OutboxMessage -> COMMIT PostgreSQL -> 201 Created
      |
      v
      dispatcher publica depois
```

O que a Outbox persiste

`outbox_messages` guarda a identidade e o envelope necessários para publicar: `OrderId`, `MessageId` único, `CorrelationId`, `Subject`, `ContentType`, `PayloadJson`, criação, tentativas, último erro, `PublishedAtUtc` e `traceparent/tracestate`. O payload continua sendo somente `OrderCreated`; os demais campos são metadados de transporte e operação.

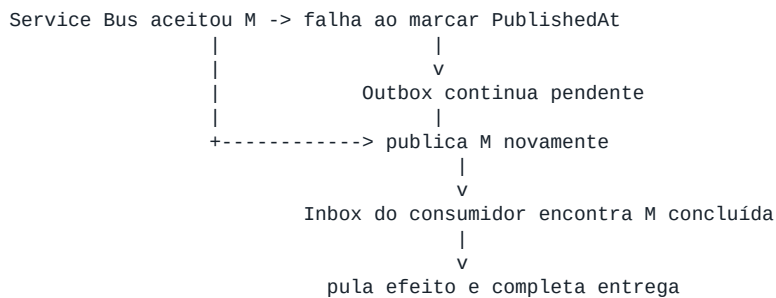
O `MessageId` é criado ao montar a mensagem de saída e reutilizado em toda tentativa. Isso é essencial: uma tentativa posterior não representa um novo evento de negócio. `AttemptCount` conta tentativas persistidas de publicação da Outbox; `DeliveryCount` é contador de entrega da subscription Service Bus. Nenhum dos dois substitui `MessageId`.

Dispatcher hospedado

`OutboxDispatcherHostedService` é hospedado na API. Ele tenta executar ao iniciar e depois pesquisa registros pendentes, em batches configuráveis, no intervalo configurável. Pendente significa `PublishedAtUtc` nulo. Ao enviar com sucesso, ele chama a operação de marcar publicado; em falha de envio, grava metadados de tentativa/erro e mantém a mensagem pendente. Não há backoff avançado, limite de tentativa ou tratamento de poison Outbox implementados ainda.

```
outbox_messages (PublishedAtUtc = null)
  |
  v
Dispatcher: busca batch pendente
  |
  v
Service Bus aceita a mensagem?
  |
  não      |      sim
  |         |      |
registra tentativa/erro  marca PublishedAtUtc
mantém pendente          |
  |                       v
+----- não consulta novamente
```

Existe uma janela inevitável: Service Bus pode aceitar a mensagem e o processo falhar antes de `PublishedAtUtc` ser confirmado no banco. Ao reiniciar, o dispatcher a vê pendente e pode publicar de novo com o mesmo `MessageId`. Logo, Outbox dá publicação at-least-once, não exactly-once. O Inbox dos consumidores transforma essa duplicata possível em efeito interno único por consumidor.



O dispatcher atual não coordena múltiplas instâncias. Não há claims, leases nem `FOR UPDATE SKIP LOCKED`; por isso a operação exige **um único dispatcher ativo**. Em uma topologia futura com mais de uma API, réplicas adicionais devem estar com `Outbox:Enabled=false` até existir coordenação. O código não impõe isso automaticamente. Limpeza/retenção de mensagens publicadas, replay e política para poison Outbox também são decisões explicitamente adiadas.

Key takeaways

- Outbox torna atômica a criação de Order e intenção de publicar, dentro de PostgreSQL.
- 201 Created significa commit durável local, não mensagem já entregue no Service Bus.
- Publicação duplicada continua possível; `MessageId` estável e Inbox reduzem seu efeito.

Common mistakes

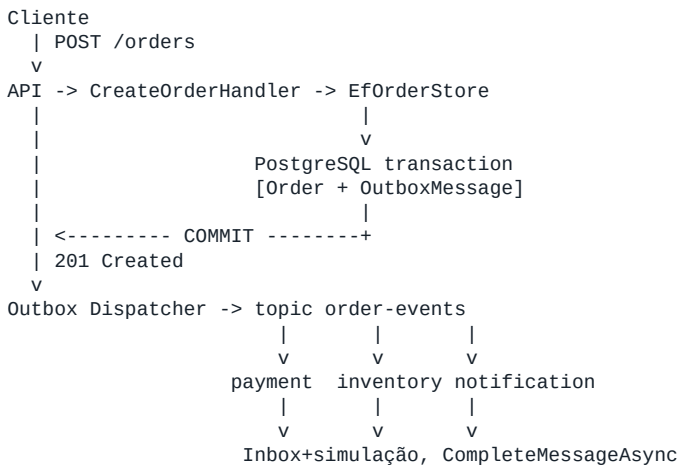
- Afirmar que Outbox cria transação única entre PostgreSQL e Service Bus.
- Marcar publicação antes do broker aceitar a mensagem.
- Escalar o dispatcher atual horizontalmente sem coordenação.

Revisão

- 1 **Qual é o limite transacional exato?** Order + `OutboxMessage` no PostgreSQL.
- 2 **Por que uma mensagem enviada pode continuar pendente?** Porque o processo pode falhar entre o aceite do broker e o commit de `PublishedAtUtc`.
- 3 **O que falta para múltiplos dispatchers?** Coordenação de claims/leases ou locking seguro entre instâncias.

9. O sistema inteiro: garantias e evolução

Um pedido bem-sucedido, passo a passo



- 1 A API recebe POST /orders e delega ao handler.
- 2 O handler cria Order em Pending, monta OrderCreated e os metadados de saída. CorrelationId recebe o OrderId; o MessageId nasce estável para a Outbox.
- 3 EfOrderStore confirma Order e OutboxMessage na mesma transação PostgreSQL.
- 4 Só então a API devolve 201 Created.
- 5 O dispatcher coleta a mensagem pendente e a publica em order-events.
- 6 Cada subscription recebe seu fluxo independente.
- 7 Cada worker valida contrato, tenta a reivindicação de Inbox de sua própria identidade, executa a simulação uma vez e completa a entrega.
- 8 Traces, métricas e logs conectam as fases sem inserir IDs técnicos no payload de negócio.

Falhas que mudam o raciocínio

- 1. PostgreSQL falha durante criação.** A transação não confirma. Não existe pedido nem Outbox durável, e a API não responde 201. Não há publicação posterior legítima. Este é o ganho da fronteira atômica local.
- 2. Service Bus indisponível no dispatcher.** O pedido e a Outbox já existem. O dispatcher registra a tentativa/erro e mantém a linha pendente; em uma execução posterior tenta de novo. A resposta original ao cliente não é desfeita, pois a intenção foi persistida.
- 3. Send aceito, mark-published falha.** A linha permanece pendente; uma nova tentativa pode duplicar o MessageId. O broker pode entregar a duplicata; Inbox evita repetir o efeito interno de cada consumidor. Não há exactly-once de publicação.
- 4. Consumidor falha repetidamente no banco/regra transitória.** Sem completion, o broker faz redelivery. Ao exceder MaxDeliveryCount provisionado, a mensagem vai para DLQ com a razão do sistema. Violação permanente de contrato e status não suportado seguem à DLQ imediatamente com razões específicas.

Padrões e problema que cada um resolve

Padrão / mecanismo	Problema principal que resolve
Boundary layers	acoplamento entre negócio e adaptadores
Migrations explícitas	evolução deliberada do schema
Topic + subscriptions	fan-out independente
PeekLock + completion	ack somente após processamento
Inbox	efeito local duplicado no consumidor
Retry/redelivery + DLQ	falhas transitórias e mensagens poison
Transactional Outbox	perda entre commit do pedido e publish
OpenTelemetry	diagnóstico transversal e operação

Os padrões se complementam, não se substituem. Outbox não impede duplicata no consumidor; Inbox não garante que o evento sairá do banco; retry não corrige contrato impossível; DLQ não é workflow de negócio; request idempotency, se vier a ser necessária, resolve repetição de POST pelo cliente e é diferente de Inbox. CloudOrders não implementa hoje uma camada explícita de idempotência de requisição HTTP.

Por que a ordem de implementação importou

Primeiro veio persistir um pedido de verdade. Depois, a probe comprovou o transporte. Depois, OrderCreated apresentou dual-write e exigiu compreender entrega assíncrona. Os workers materializaram fan-out. Duplicatas e settlement expuseram a necessidade de Inbox. Falhas repetidas deram sentido a redelivery e DLQ. Observabilidade tornou essas fases legíveis. A Outbox encerrou a lacuna mais importante entre decisão durável e publicação. Construir tudo de uma vez teria escondido por que cada mecanismo existe.

Próxima fase: production scale e deploy-to-Azure

Uma evolução de produção precisaria, entre outros pontos, de coordenação segura de dispatcher para múltiplas instâncias, retenção e limpeza de Outbox, administração/replay governado de DLQ, políticas de retry/backoff mais maduras, provedores externos idempotentes e reconciliação, alertas/dashboards/OTLP centralizado, gestão de segredos e capacidade medida. Essas mudanças evoluem a operação; não pedem redesenhar o contrato OrderCreated, a direção das dependências ou a complementaridade Inbox+Outbox.

Deployment em Azure deverá adicionar provisionamento do Service Bus e banco gerenciado, identidade/segredos, configurações por ambiente, observabilidade centralizada e automação de entrega. CI/CD e IaC também pertencem a essa próxima fase. Não devem ser descritos como presentes, nem justificar uma mudança prematura da arquitetura implementada.

Key takeaways

- A atomicidade atual é local ao PostgreSQL; publicação e consumo são assíncronos e at-least-once.
- Inbox, Outbox, broker retry/DLQ e futura request idempotency cobrem problemas diferentes.
- Evoluir para Azure muda operação e escala; não invalida os limites que o walking skeleton estabeleceu.

Common mistakes

- Dizer que 201 prova consumo pelos três workers.
- Confundir publicação eventual com perda inevitável.
- Tratar recurso manual documentado como ambiente Azure já implantado.

Revisão

- 1 **O que `201 Created` garante?** O commit PostgreSQL do pedido e da Outbox, não a publicação/consumo imediato.
- 2 **Por que há duplicata após `send` bem-sucedido?** O registro de publicação pode falhar depois do aceite do broker.
- 3 **Qual padrão resolveria `POST` duplicado de cliente?** Idempotência de requisição, distinta do Inbox de consumidor e não implementada atualmente.

10. Referência para revisão

Mental model

Pense no CloudOrders como uma sequência de compromissos locais confiáveis conectados por comunicação assíncrona. O pedido não tenta manter uma transação global com o broker. Ele confirma decisão e intenção de publicar em PostgreSQL; o dispatcher converte a intenção em mensagem; o broker tenta entregar; cada consumidor transforma a entrega em efeito local único para si. Onde o sistema não consegue prometer uma transação global, ele registra estado, aceita repetição e torna a repetição segura.

Glossário

- **At-least-once:** entrega/publicação pode ocorrer uma ou mais vezes; consumidor deve tolerar duplicatas.
- **CorrelationId:** identidade de correlação no transporte; em OrderCreated, é o OrderId em formato D.
- **Dead-letter queue (DLQ):** fila de mensagens que não progridem por falha permanente ou excesso de tentativas.
- **DeliveryCount:** número de entregas do broker para uma mensagem em uma subscription; não é identidade.
- **Dispatcher:** serviço hospedado que lê Outbox pendente e tenta publicar.
- **Exactly-once:** efeito uma única vez em todo o sistema; não é garantia do CloudOrders.
- **Fan-out:** uma publicação chega a múltiplos consumidores independentes por subscriptions.
- **Inbox:** registro transacional de processamento por consumidor que evita repetir efeito local.
- **MessageId:** identidade estável da mensagem lógica no envelope Service Bus e na Outbox.
- **Outbox:** registro transacional da intenção de publicar, persistido com a mudança de negócio.
- **PeekLock:** modo no qual o broker mantém a mensagem bloqueada até completion ou expiração do lock.
- **Poison message:** mensagem que não progride sem investigação ou mudança.
- **Redelivery:** nova entrega pelo broker após ausência/falha de settlement.
- **TraceId:** identidade técnica W3C de uma trace de observabilidade.

Decisões arquiteturais resumidas

Decisão	Motivo	Limite assumido
PostgreSQL + migrations explícitas externa ao startup	transação/schema real e controlado	operação de migration é
Tópico order-events e subscriptions	fan-out de eventos de pedido	requer Service Bus Standard
AutoComplete=false + PeekLock redeliverar	concluir só após processar	settlement pode falhar e
Inbox composto	uma vez por consumidor	não dá exactly-once externo
Sem Polly local adiadas	broker governa redelivery	políticas avançadas ficam
Outbox transacional	não perder intenção após commit	pode publicar duplicado
Um dispatcher ativo ainda	não há coordenação distribuída hoje	não escala horizontalmente
OpenTelemetry	investigar fluxo distribuído	exige destino/alertas para
operação madura		

Failure-mode cheat sheet

Sintoma

POST não retorna 201 por erro PostgreSQL
Pedido existe, evento ainda não chegou
Mensagem aparece duas vezes
Uma subscription falha, outra conclui
Contrato inválido na worker
Status não Pending
Falha transitória repetida
Auth PostgreSQL falha
Tabela ausente
Sem span do Service Bus

Primeiro raciocínio seguro

pedido e Outbox não devem ter confirmado
investigar Outbox pendente/dispatcher/broker
comparar MessageId; Inbox deve suprimir efeito repetido
esperado: fan-out é independente
DLQ manual: MessageContractViolation
DLQ manual: UnsupportedBusinessStatus
redelivery; depois MaxDeliveryCountExceeded pelo broker
conferir origem da configuração e volume, sem expor segredo
conferir migration e schema aplicado
conferir feature flag e configuração antes do startup

Cheat sheet de IDs e contadores

Campo / contador

OrderId

MessageId

CorrelationId

TraceId

AttemptCount

DeliveryCount

Pergunta que responde

Qual pedido de negócio?

Qual mensagem lógica?

A que pedido a mensagem se correlaciona?

Qual execução técnica percorreu o fluxo?

Quantas tentativas a Outbox persistiu?

Quantas entregas a subscription fez?

Escopo

domínio

Outbox/envelope, estável

transporte; OrderId em formato D

observabilidade W3C

Outbox

Service Bus

Perguntas e respostas de revisão final

- 1 Por que não publicar diretamente depois de gravar o pedido?** Porque uma falha no broker após o commit perde o evento; Outbox persiste a intenção no mesmo commit.
- 2 Outbox elimina todas as duplicatas?** Não. Ela aceita publicação at-least-once; Inbox lida com efeitos internos duplicados.
- 3 O que um tópico oferece que uma queue não oferece?** Cópias lógicas independentes para várias subscriptions.
- 4 Por que o payload não contém TraceId?** Trace context é técnico e evolui separadamente do contrato de negócio.
- 5 Por que não completar antes do banco?** Porque o broker consideraria entregue uma ação que talvez não tenha sido concluída.
- 6 O que impede payment de processar duas vezes?** A reivindicação atômica da Inbox para (ConsumerName, MessageId).
- 7 Por que inventory pode processar o mesmo MessageId?** Porque é outro consumidor, com efeito próprio e legítimo.
- 8 O que ocorre ao falhar CompleteMessageAsync?** Pode haver redelivery; Inbox já concluída faz o worker evitar o efeito repetido.
- 9 Qual falha vai diretamente à DLQ?** Violação de contrato ou status de negócio não suportado conhecidos.
- 10 Qual falha redelivera primeiro?** Falha transitória de processamento ou infraestrutura sem settlement.
- 11 O que MaxDeliveryCount=5 significa aqui?** Política de provisionamento documentada para subscriptions de negócio; após o limite o broker DLQ.
- 12 O que não é seguro fazer com o dispatcher atual?** Ter várias instâncias ativas sem claims/leases/locking de coordenação.
- 13 Como distinguir erro de schema de erro de senha?** O primeiro ocorre após conectar e consulta schema; o segundo impede autenticação/conexão válida.
- 14 Por que testes com Compose não bastam?** Um banco manual pode carregar estado e migration que mascaram falhas; Testcontainers cria ambiente limpo.

- 15 **O que ainda falta para produção?** Coordenação, retenção/replay, provedores reais idempotentes, IaC, CI/CD, segredos e telemetria centralizada.

Cenários de entrevista e linha de raciocínio

- 1 **“O cliente recebeu 201, mas payment não rodou. O pedido se perdeu?”** Não conclua isso. 201 prova o commit de Order+Outbox. Verifique a linha pendente, tentativas/erro do dispatcher, publicação e a subscription. A arquitetura escolhe publicação eventual.
- 1 **“Vemos duas mensagens com o mesmo MessageId em payment.”** Investigue a janela entre send e mark-published. É comportamento possível da Outbox at-least-once. Verifique se a Inbox de Payment registrou apenas um efeito; não gere novo MessageId como correção.
- 1 **“Quero rodar dez APIs para aumentar disponibilidade.”** O dispatcher atual não coordena concorrência. Mantenha um ativo ou implemente leases/claims/SKIP LOCKED antes de escalar essa parte. Réplicas adicionais devem desabilitar Outbox: Enabled até isso existir.
- 1 **“Uma mensagem com JSON inválido falhou cinco vezes.”** Isso é desperdício de tentativas para uma falha permanente. A validação deve classificá-la como MessageContractViolation e enviá-la à DLQ imediatamente, com diagnóstico seguro.
- 1 **“A charge externa ocorreu, mas o worker caiu antes do commit local.”** Inbox local não prova que a charge não repetirá. O provedor real precisará de chave idempotente/reconciliação e talvez estado adicional; esse limite não deve ser escondido.
- 1 **“Métricas de runtime aparecem, mas não há mensagens processadas.”** Métricas de runtime apenas mostram processo ativo. Procure a Activity/métrica de processamento, a subscription, filtros, configuração e a feature flag do SDK para spans Service Bus.
- 1 **“Mudei a senha no Compose e o banco recusou conexão.”** Considere que o volume existente preservou a credencial de inicialização. Verifique a fonte efetiva da configuração e reinicialize/ajuste o ambiente de modo consciente, sem expor valor secreto.
- 1 **“Vamos colocar OrderId como tag em todas as métricas.”** Rejeite: cardinalidade explode. Use logs/traces para IDs e métricas agregadas para taxas, duração e erros.
- 1 **“Podemos chamar CompleteMessageAsync no começo para ganhar throughput?”** Não; uma falha posterior perderia a oportunidade de redelivery. Throughput deve ser evoluído com concorrência, medição e idempotência, preservando ack após sucesso.
- 1 **“A Outbox torna PostgreSQL e Service Bus uma única transação?”** Não. Ela transforma uma lacuna distribuída em estado durável e reprocessável, aceitando publicação duplicada e usando Inbox para conter o efeito no consumo.

Final end-to-end: o que acontece depois de POST /orders

Após um POST /orders válido, a API delega a criação ao caso de uso. O pedido Pending, o evento OrderCreated e uma OutboxMessage com metadados estáveis são construídos. A transação PostgreSQL confirma pedido e intenção de publicação; então vem 201 Created. Em outro momento — possivelmente imediatamente, possivelmente após uma falha recuperável — o dispatcher lê a Outbox pendente e publica no tópico. O broker replica logicamente a publicação às subscriptions de Payment, Inventory e Notification. Cada worker valida o contrato, reivindica sua própria Inbox, executa sua simulação determinística uma vez e completa a mensagem. Se algo falhar, observabilidade ajuda a localizar a fase, o broker pode redeliverar e a DLQ isola mensagens que não progridem. Esse é o desenho: decisões locais duráveis, comunicação assíncrona, repetição esperada e efeitos locais protegidos.

Base documental e comando de geração

Este guia foi consolidado a partir dos capítulos 01 a 10, do índice de estudo, do `README.md` da raiz, das ADRs em `docs/decisions`, dos contratos canônicos em `openspec/specs` e dos changes arquivados desde `bootstrap-cloudorders-walking-skeleton` até `add-outbox-pattern`. A conferência final deve sempre prevalecer sobre o código e contratos atuais quando o sistema evoluir.

Para reproduzir o PDF com o runtime de Python que tenha `reportlab` disponível, execute na raiz do repositório:

```
python docs/study/build-study-guide-pdf.py docs/study/CloudOrders-Study-Guide.md docs/study/CloudOrders-Study-Guide.pdf
```